SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1410 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS:4 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I K.YOGANAND/192525173 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
K.YOGANAND

Course Faculty
Dr.N.Deepa

Annexure A

DESIGN TASK

Q1. Predictive Parser Design (Application Level)

Task:

Construct an LL(1) predictive parsing table for the given grammar and demonstrate its application by parsing the specified input string step-by-step, clearly showing stack operations, input symbols, and parsing actions.

Grammar:

E → T E'

E' → + T E' | ε

T → F T'

T' → * F T' | ε

F → (E) | id

Conditions:

- Remove left recursion (if any)
- Compute FIRST and FOLLOW sets
- Construct LL(1) parsing table **Test Case: Input String:** id + id * id

Expected Output:

- Step-by-step parsing actions
- Final result: **String Accepted** **Solution:**

Q1. Predictive Parsing (LL(1))

1. Given grammar

$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow * FT' \mid \varepsilon$$
$$F \rightarrow (E) \mid id$$

2. Remove left recursion

The given grammar has no left recursion. It is already suitable for LL(1) predictive parsing.

3. FIRST sets

Non-terminal	FIRST
E	{(, id)}
E'	{+, varepsilon}

T	{(, id}
T'	{*, varepsilon}
F	{(, id}

4. FOLLOW sets

Non-terminal	FOLLOW
E	{), $}
E'	{), $}
T	{+,), $}
T'	{+,), $}
F	{*, +,), $}

Here, $ represents the end of the input.

5. LL(1) predictive parsing table

Non-terminal	id	+	*	(	)	$
E	TE'	—	—	TE'	—	—
E'	—	+TE'	—	—	Varepsilon	varepsilon
T	FT'	—	—	FT'	—	—
T'	—	varepsilon	*FT'	—	varepsilon	Varepsilon
F	id	—	—	(E)	—	—

— means an error or empty table entry.

6. Step-by-step parsing

Input string: id + id * id

Initial stack: $ E (top is on the right). The input is followed by $.

Step	Stack (top right)	Input	Action
1	$ E	id + id * id $	E rightarrowTE'
2	$ E' T	id + id * id $	T rightarrow FT'
3	$ E' T' F	id + id * id $	F rightarrow id
4	$ E' T' id	id + id * id $	Match id
5	$ E' T'	+ id * id $	T' rightarrow varepsilon
6	$ E'	+ id * id $	E' rightarrow +TE'
7	$ E' T +	+ id * id $	Match +
8	$ E' T	id * id $	T rightarrow FT'
9	$ E' T' F	id * id $	F rightarrow id
10	$ E' T' id	id * id $	Match id

11	$ E' T'	* id $	T' rightarrow *FT'
12	$ E' T' F *	* id $	Match *
13	$ E' T' F	id $	F rightarrow id
14	$ E' T' id	id $	Match id
15	$ E' T'	$	T'rightarrow\varepsilon
16	$ E'	$	E' rightarrow varepsilon
17	$	$	Accept

Final result: String Accepted.

Q2. Shift-Reduce Parsing (Application Level) Task:

Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly indicating all reductions and decisions made during parsing.

Grammar:

E → E + E

E → E * E

E → id

Conditions:

- Show stack, input buffer, and action at each step
- Resolve conflicts using precedence (* has higher precedence than +*)

Test Case: Input:

id + id * id

Expected Output:

- Sequence of actions: Shift / Reduce / Accept
- Final result: **String Accepted**

Integrated Design Question (SLR, LALR, CLR Parsing) **Solution:**

1. Given grammar

$$E \to E + E$$
$$E \to E * E$$
$$E \to id$$

2. Step-by-step parsing

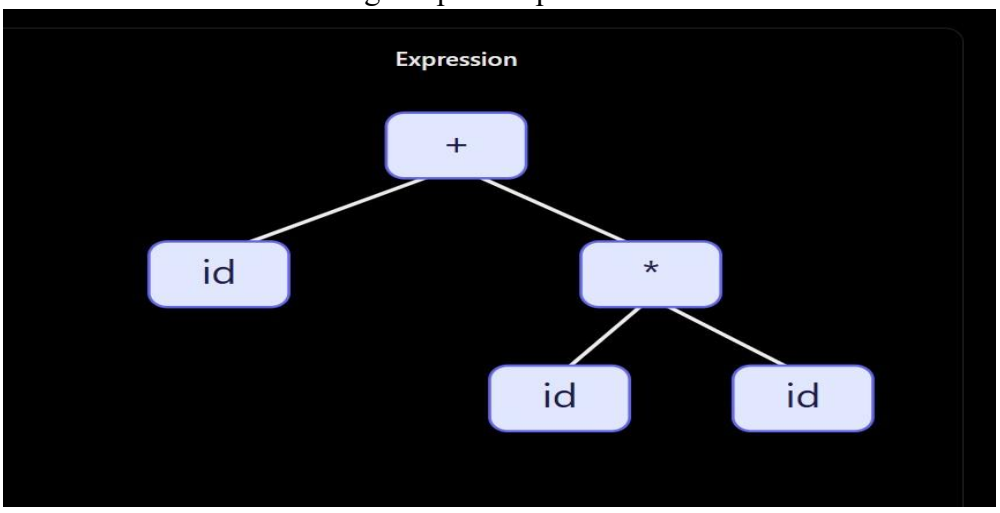
The stack starts empty. The input buffer ends with $.

Step	Stack	Input buffer	Action
1	$	id + id * id $	Shift id
2	$ id	+ id * id $	Reduce \(E\rightarrow id\)
3	$ E	+ id * id $	Shift +

4	$ E +	id * id $	Shift id
5	$ E + id	* id $	Reduce \(E\rightarrow id\)
6	$ E + E	* id $	Shift * because it has higher precedence
7	$ E + E *	id $	Shift id
8	$ E + E * id	$	Reduce \(E\rightarrow id\)
9	$ E + E * E	$	Reduce \(E\rightarrow E*E\)
10	$ E + E	$	Reduce \(E\rightarrow E+E\)
11	$ E	$	Accept

3. Parse structure

The expression is evaluated according to operator precedence:



Multiplication is grouped before addition.

The reductions correspond to:

$$1.\ E \rightarrow id$$

$$2.E \rightarrow id$$

$$3.E \rightarrow id$$

$$4.E \rightarrow E * E$$

$$5.E \rightarrow E + E$$

Final result: String Accepted.

Q3.Construct the LR parsing tables for the given grammar and perform a detailed comparison of different LR parsing techniques (SLR, LALR, and CLR), highlighting their construction steps, state transitions, table sizes, and any conflicts encountered.

S → L = R | R

L → * R | id

R → L

Tasks

(i) CLR (Canonical LR) Design

Construct the canonical collection of LR(1) items

Build the CLR parsing table

Show all item sets and transitions

(ii) SLR Parser Design

Compute FIRST and FOLLOW sets

Construct the SLR parsing table

Identify and explain any shift-reduce or reduce-reduce conflicts

(iii) LALR Parser Design

Merge compatible LR(1) states

Construct the LALR parsing table

Compare with CLR and show how states are reduced

(iv) Parsing Demonstration

Using any one of the constructed parsing tables, parse

the input string:

id = * id Show:

Stack

Input buffer

Action (Shift / Reduce / Accept) **Solution:**

1. Given grammar

$$S \rightarrow L = R \mid R$$
$$L \rightarrow * R \mid id$$
$$R \rightarrow L$$

Number the productions as follows.

Production number	Production
(0)	$S' \rightarrow S$
(1)	$S \rightarrow L = R$
(2)	$S \rightarrow R$
(3)	$L \rightarrow * R$
(4)	$L \rightarrow id$
(5)	$R \rightarrow L$

2. FIRST and FOLLOW sets

FIRST sets

Non-terminal	FIRST
S	{*, id}
L	{*, id}
R	{*, id}

FOLLOW sets

Non-terminal	FOLLOW
S	{$}
L	{=,$}
R	{=,$}

These FOLLOW sets are used to construct the SLR parsing table.

3. **CLR (Canonical LR(1)) design**

CLR uses LR(1) items containing a production, a dot position, and a lookahead symbol. An item such as

$$[L \rightarrow \cdot\, id, =]$$

means that the parser expects id while the lookahead is =.

Canonical LR(1) item sets

The following item sets use the notation [production, lookahead]. Where two lookaheads occur for the same production and dot position, they are shown separately or grouped.

Initial state \(I_0\)

$$[S' \rightarrow \cdot\, S, \$]$$
$$[S \rightarrow \cdot\, L = R, \$]$$
$$[S \rightarrow \cdot\, R, \$]$$

$$[L \rightarrow \cdot * R, \{=, \$\}]$$
$$[L \rightarrow \cdot\, id, \{=, \$\}]$$
$$[R \rightarrow \cdot\, L, \$]$$

Remaining states

Each state is obtained by moving the dot over one grammar symbol and applying closure.

I_1

$$[S' \rightarrow S\cdot, \$]$$

I_2

$$[S \rightarrow L\cdot = R, \$]$$
$$[R \rightarrow L\cdot, \$]$$

I_3

$$[S \rightarrow R\cdot, \$]$$

I_4

$$[L \rightarrow * \cdot R, \{=, \$\}]$$
$$[R \rightarrow \cdot L, \{=, \$\}]$$
$$[L \rightarrow \cdot * R, \{=, \$\}]$$
$$[L \rightarrow \cdot id, \{=, \$\}]$$

I_5

$$[L \rightarrow id\cdot, \{=, \$\}]$$

I_6

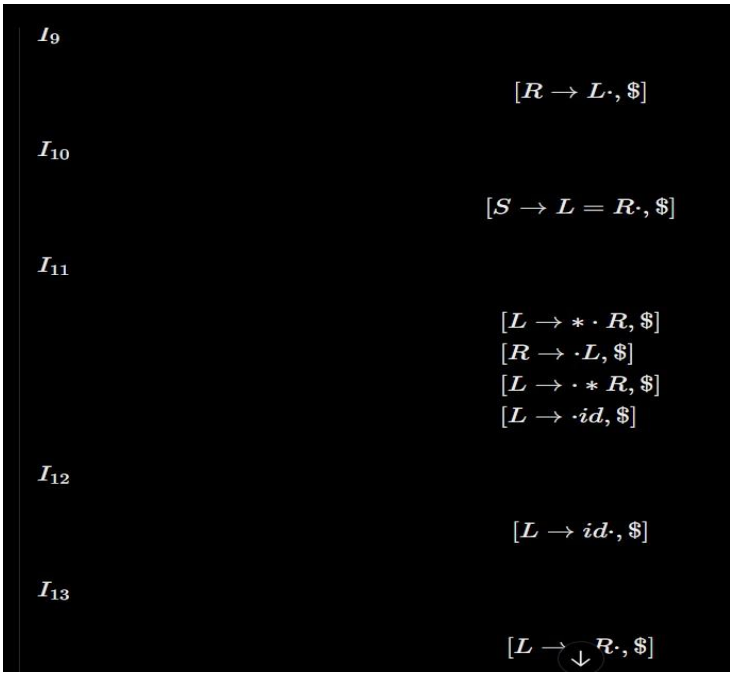
$$[S \rightarrow L = \cdot R, \$]$$
$$[R \rightarrow \cdot L, \$]$$
$$[L \rightarrow \cdot * R, \$]$$
$$[L \rightarrow \cdot id, \$]$$

I_7

$$[R \rightarrow L\cdot, \{=, \$\}]$$

I_8

$$[L \rightarrow *R\cdot, \{=, \$\}]$$



CLR state transitions

State	Symbol	Next state
I_0	S	I_1
I_0	L	I_2
I_0	R	I_3
I_0	*	I_4
I_0	id	I_5
I_2	=	I_6
I_4	L	I_7
I_4	R	I_8
I_4	*	I_4
I_4	id	I_5
I_6	L	I_9
I_6	R	I_{10}
I_6	*	I_{11}
I_6	id	I_{12}
I_{11}	L	I_9
I_{11}	R	I_{13}
I_{11}	*	I_{11}
I_{11}	id	I_{12}

All other transitions are undefined.

CLR parsing table

Notation: s5 = shift to state 5; r4 = reduce by production (4); acc = accept.

State	id	*	=	$	GOTO S	GOTO L	GOTO R
0	s5	s4	—	—	1	2	3
1	—	—	—	acc	—	—	—
2	—	—	s6	r5	—	—	—
3	—	—	—	r2	—	—	—
4	s5	s4	—	—	—	7	8
5	—	—	r4	r4	—	—	—
6	s12	s11	—	—	—	9	10
7	—	—	r5	r5	—	—	—
8	—	—	r3	r3	—	—	—
9	—	—	—	r5	—	—	—
10	—	—	—	r1	—	—	—
11	s12	s11	—	—	—	9	13
12	—	—	—	r4	—	—	—
13	—	—	—	r3	—	—	—

CLR result: No shift-reduce or reduce-reduce conflicts.

4. SLR parser design

SLR uses LR(0) item sets and the FOLLOW sets to determine where reductions are permitted.

SLR parsing table

State	id	*	=	$	GOTO S	GOTO L	GOTO R
0	s5	s4	—	—	1	2	3
1	—	—	—	acc	—	—	—
2	—	—	s6/r5	r5	—	—	—
3	—	—	—	r2	—	—	—
4	s5	s4	—	—	—	7	8
5	—	—	r4	r4	—	—	—
6	s12	s11	—	—	—	9	10
7	—	—	r5	r5	—	—	—

8	—	—	r3	r3	—	—	—
9	—	—	—	r5	—	—	—
10	—	—	—	r1	—	—	—
11	s12	s11	—	—	—	9	13
12	—	—	r4	r4	—	—	—
13	—	—	r3	r3	—	—	—

Conflict explanation

In state 2, the items are:

$$S \to L \cdot = R$$
$$R \to L \cdot$$

- The first item requires a shift on =.
- The second item requires a reduce by \(R\rightarrow L\).
- Since = belongs to FOLLOW(\(R\)), SLR places both actions in the same cell.

Therefore, SLR has one shift-reduce conflict at state 2 on =.

5. LALR parser design

LALR merges CLR states that have identical LR(0) cores and combines their lookahead sets.

For this grammar, the compatible state pairs are:

- I_4 and I_{11}
- I_5 and I_{12}
- I_7 and I_9
- I_8 and I_{13}

Thus, the number of states decreases from 14 CLR states to 10 LALR states.

LALR parsing table

The following table uses renumbered merged states: state 0 is the initial state, and the other states are numbered according to the merged transitions.

State	id	*	=	$	GOTO S	GOTO L	GOTO R
0	s5	s4	—	—	1	2	3
1	—	—	—	acc	—	—	—
2	—	—	s6	r5	—	—	—
3	—	—	—	r2	—	—	—
4	s5	s4	—	—	—	7	8
5	—	—	r4	r4	—	—	—
6	s5	s4	—	—	—	9	10
7	—	—	r5	r5	—	—	—

8	—	—	r3	r3	—	—	—
9	—	—	—	r1	—	—	—
10	—	—	—	r3	—	—	—

LALR result: No parsing conflicts occur for this grammar.

6. Comparison of SLR, LALR and CLR

Feature	**SLR(1)**	**LALR(1)**	**CLR(1)**
Items used	LR(0)	Merged LR(1)	LR(1)
Reduction lookaheads	FOLLOW sets	Individual LR(1) lookaheads, merged	Individual LR(1) lookaheads
Table size	Small	Medium	Large
States this for grammar	10 LR(0) states	10 merged states	14 states
Conflicts here	One shift-reduce conflict	None	None
Parsing power	Lowest of these three	Greater than SLR in general	Greatest of these three

Note: SLR has 10 LR(0) states; its table is not directly interchangeable with the CLR table because the reduction rules differ.

7. Parsing demonstration using CLR

Input string: id = * id $

Using the CLR table from Section 3:

Step	**Stack**	**Input buffer**	**Action**
1	0	id = * id $	Shift id, state 5
2	0 id 5	= * id $	Reduce $L \rightarrow id$
3	0 L 2	= * id $	Shift =, state 6
4	0 L 2 = 6	* id $	Shift *, state 11
5	0 L 2 = 6 * 11	id $	Shift id, state 12
6	0 L 2 = 6 * 11 id 12	$	Reduce $L \rightarrow id$
7	0 L 2 = 6 * 11 L 9	$	Reduce $R \rightarrow L$
8	0 L 2 = 6 * 11 R 13	$	Reduce $L \rightarrow * R$
9	0 L 2 = 6 R 10	$	Reduce $S \rightarrow L = R$

10	0 S 1	$	Accept

Final result

String Accepted.

The three parsing methods differ in how they construct states and decide reductions. For this grammar, CLR and LALR parse without conflicts, while SLR encounters a shift-reduce conflict because it uses the broader FOLLOW sets.

[R → L., $]

20/10/26

CO2 — AT1

K. Yoganand
192528173

CIA-1410 Compiler Design
slot D

6 Comparative Analysis of SLR vs LALR vs CLR

Given Grammar

$S \rightarrow L = R / R$
$L \rightarrow *R / id$
$R \rightarrow L$

Augmented Grammar: $S' \rightarrow S$

production numbers

NO	production
1	$S \rightarrow L = R$
2	$S \rightarrow R$
3.	$L \rightarrow *R$
4.	$L \rightarrow id$
5.	$R \rightarrow L$

Follow sets for SLR parsing :-

first the follow sets are calculated from the Grammer

FOLLOW (S) = { $ }
FOLLOW (L) = { =, $ }
FOLLOW (R) = { =, $ }

LR(0) item sets :-

-the Canonical LR(0) collection contains 10 states, To t

I_0 : $S' \to .S$
$S \to .L=R$
$S \to .R$
$L \to .*R$
$L \to .id$
$R \to .L$

I_1 : $S' \to S.$

I_2 : $S \to L.=R$
$R \to L.$

I_3 : $L \to *.R$
$R \to .L$
$L \to .*R$
$L \to .id$

I_4 : $L \to id.$

I_5 : $S \to L=.R$
$R \to .L$
$L \to .*R$
$L \to .id$

I_6 : $L \to *R$

I_7 : $R \to L$

I_8 : $S \to L=R$

* LR(0) transitions :-

state	symbol	next state
I_0	S	I_1
I_0	L	I_2
I_0	R	I_3
I_0	*	I_4
I_0	id	I_5
I_0	=	
I_2	R	I_6
I_3	L	I_7
I_3	*	I_3
I_3	id	I_4
I_4	R	I_8
I_5	L	I_7
I_5	*	I_3
I_5	id	I_4
I_6	R	I_9
I_6	L	I_7
I_6	*	I_3
I_6	id	I_4

∴ the Grammer is not SLR(1)

* CLR (Canonical LR(1) parsing) :-

$R \to L.$
$[R \to L., \$]$

[S → L.=R, $]
[R → L., $]

⇒ CLR shifts on = and reduces on $ eliminating the SLR conflict

LALR parsing :-

state	id	*	=	$
0	S_5	S_4	–	–
1	–	–	–	acc
2	–	–	S_6	r_5
3	–	–	–	r_2
4	S_5	S_4	–	–
5	–	–	r_4	r_5
6	S_5	S_4	–	–
7	–	–	r_3	r_3
8	–	–	r_5	r_5
9	–	–	–	r_1

* Test Case : id = * id

- the input is valid because it can be derived as follows :

S ⇒ L=R
⇒ id=R
⇒ id=L
⇒ id=*R
⇒ id=*L
⇒ id=*id

* Comparison of SLR, LALR, & CLR

Feature	SLR (1)	LALR (1)	CLR (1)
Items used	LR(0)	LR(1) cores merged	FULL LR(1)
Look a head	Follow sets	Item specific	Item specific
states	10	10	more than 10
conflicts for this G	shift/ Reduce	none	none
Table size	small	small	Largest
parsing power	lowest of 3	↑ than SLR	↑ of the three

7. Operator precedence parsing design and evaluation

Given Grammer

$E \rightarrow E+E \mid E*E \mid (E) \mid id$

input : id + id * id

Operator precedence relation :-

Relation	Meaning
$a < b$	shift the next input symbol
$a = b$	Equal precedence boundary, such as matching parenthesis
$a > b$	Reduce the symbols on the stack

* Operator precedence table :-

stack top / input	id	+	*	(	)	$
id	-	>	>	-	>	>
+	<	>	<	<	>	>
*	<	>	>	<	>	>
(	<	<	<	<	=	-
)	-	>	>	-	>	>
$	<	<	<	<	-	Accept

Step by step parsing :-

input : id + id * id $

step	stack	input	Action
1	$	id + id * id $	shift id
	$ id	+ id * id $	Reduce E → id
	$ E	+ id * id $	shift +
	$ E +	id * id $	shift id
	$ E + id	* id $	E → id
	$ E + E	* id $	shift *
	$ E + E *	id $	shift id
	$ E + E * id	$	E → id
	$ E + E * E	$	E → E * E
	$ E + E	$	E → E + E
	$ E	$	Accept

-* Conflict detection and resolution :-

-the Grammar

$$e \rightarrow e+e \mid e*e \mid (e) \mid id$$

Correctness Evaluation :-

Criterion	Result
Multiplication precedence	Correct
Addition precedence	Correct
Left Associativity	Enforced
parentheses handling	supported
Invalid input detection	undefined values signal errors
Test Input	Accepted

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis

Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation